

Last modified on

Sports Data Soccer API - Match Player Ratings (MA20)

Get player ratings for a match based on a combination of attacking, defending, goalkeeping, passing, possession and shooting.

Overview

The Soccer **Match Player Ratings feed** provides ratings for a specific match, based on totals in a variety of statistics for that player. The total score (index score) has six subcomponents (raw index scores), corresponding to different qualitative aspects of a player's performance. The total score is calculated based on each subscore and according to the player's position in the match.

For further explanation of the Index Score and a detailed description of each subscore, see: [Player Ratings explained - Subscores and Total Score](#)

The six subscores are:

- Attacking
- Defending
- Goalkeeping
- Passing
- Possession
- Shooting

Update - Behind Closed Doors and Limited Audience matches (May 2020):

We have implemented a way to highlight when a match will be played behind closed doors or with a limited audience. When these conditions apply, new attributes for `attendanceInfoId` and `attendanceInfo` will be delivered in the feed under the `matchInfo` element. For more information, see: [Feed Output](#)

Required Compliance:

To comply with our API security and best practice logic for your API calls please refer to our [FAQs](#). We also recommend that you take a look at our [API Overview and Authentication Guide](#). You will learn the basics of how our API works and key concepts to get started.

How Often Should I Poll The Feed?

We have provided some guidelines to help ensure that you call the feed only as often as you need to. Make sure that you don't exceed the rate confirmed with us - this can vary depending on whether you make requests as a B2B or B2C outlet - if you need more information on this, contact your Stats Perform representative.

UPDATED	POLLING FREQUENCY				
	TIER 15	TIER 12+14	TIER 11+13	TIER 8+9+10	TIER 6+7
Updated on stops in play within the game, including the following types of events: Goal, Shot at Goal, Free Kick, Offside Given, Cards, Start Half, End Half, Corner Won, Corner Lost, Substitution The feed should update approximately every 90 seconds and will be pushed out automatically in event of these key actions. However, there will always be a gap of at least 30 seconds between updates unless there is a goal scored, in which case an update will be produced immediately. There may be a gap of several minutes between feeds if no key events take place	Base your polling frequency on the given information - updates have a gap of at least 30 seconds and there may be a gap of several minutes between feeds if no key events take place. However, a goal update is pushed out immediately.				

Match Player Ratings (MA20) Guide

USAGE	FEED
GET match player ratings by fixture UUID (path parameter)	/soccerdata/matchpla
GET match player ratings by fixture UUID (fx parameter, also supports Opta legacy IDs - not Opta Core).	/soccerdata/matchpla

If your request URL is as follows (does not contain a fixture UUID), the feed returns an error:
/soccerdata/matchplayerratings/{outletAuthKey}

Query Parameters And Example Request Parameters

This feed also makes use of the **Global query parameters**, and you can combine them with the following query parameters (some of which can be used in conjunction with each other). However, you must query the **/matchplayerratings** feed by a specific fixture (match) UUID.

Legacy IDs: You can use the **fx** parameter to filter by legacy IDs - only one ID is supported (you cannot pass multiple IDs in a comma-separated list).

How to use query parameters and make successful requests:

In our guides, we use HTTPS syntax for our example **GET** requests (rather than cURL, for example). All URLs are case-sensitive - this means that URL strings must contain the correct case and operators. You **MUST** format your requests correctly and include certain information; otherwise, an error will be returned. You can find information about the API, including the base URL (`{protocol}://{requestDomain}/{feedResource}`), in the **main guide** and **API Overview and Authorisation guide**.

- Include the following information in the URL or header and make sure it is correct for your outlet: your unique and valid **{outletAuthKey}** (Outlet Authentication Key), query parameters for **_rt={mode}** (operating

mode) and `_fmt={dataFormat}` (response format), entity query parameter and UUID.

- Begin the query parameter string with `?` (question mark) - this must be after the Outlet Authentication Key (and endpoint, if applicable). To build up a query string, make sure that each query parameter is separated by the `&` (ampersand) operator. Values must be separated by the correct operator, for example `&` (multiple values, 'AND'), or `-` if a parameter supports it - `,` (multiple values, 'OR') or `!` ('NOT').
- Use the required `_` (underscore) for any parameters listed with this prefix.
- Remove any placeholder value curly braces `{ }` from your calls (we include these as an example only).
- If using the JSONP format (`_fmt=jsonp`) you MUST use it in combination with the callback parameter (`_clbk`) and a fixed value (or where you use a different value in a different instance, you must do this as little as possible). Be aware that common JavaScript frameworks, such as jQuery, can default to use randomly-generated function names - you must make sure that these are overridden and define a fixed function name instead.

Query parameter	Value and description
<code>{fixtureUuid}</code> Fixture UUID	GET player ratings for a match by the specified fixture UUID (path parameter method). Pass the fixture UUID in the request URL directly as the <code>{assetId}</code> (asset/resource ID). <pre>GET https://api.performfeeds.com/soccerdata/matchplayerratings/{outletAuthKey}/{fixtureUuid}?_rt={mode}&_fmt={dataFormat}</pre>
<code>fx</code>	GET player ratings data for a match by the specified fixture UUID (query parameter method) (also supports Opta legacy IDs - not Opta Core). Pass the fixture UUID to the <code>fx</code> parameter. <pre>GET https://api.performfeeds.com/soccerdata/matchplayerratings/{outletAuthKey}?_rt={mode}&_fmt={dataFormat}&fx={fixtureUuid}</pre>
<code>indexscore</code>	GET player ratings data for a match without the index score (the overall Opta player rating score). Pass a value of <code>no</code> to the <code>indexscore</code> parameter, plus the fixture UUID in the URL as the path parameter. Note: Valid values are <code>yes</code> <code>no</code> - by default, the index score is returned when the call does not include <code>indexscore</code> (equivalent to a value of <code>yes</code>). <pre>GET https://api.performfeeds.com/soccerdata/matchplayerratings/{outletAuthKey}/{fixtureUuid}?_rt={mode}&_fmt={dataFormat}&indexscore={yes no}</pre>

Sample Calls

These sample calls let you see an example URL and the response - click on the links below (hyperlinks open in a new window).

- [XML](#)
- [JSON](#)
- [XML + indexscore=no](#)
- [JSON + indexscore=no](#)

Feed Output

Here, you can find an explanation of all possible response fields and data. This is XML but fields in the feed response are similar for JSON.

▶<matchPlayerRatings> (XML only)

▶<matchInfo>

▶<description>

▶<sport>

▶<ruleset>

▶<competition>

▶<country>

▶<tournamentCalendar>

▶<stage>

▶<series>

▶<contestants> (XML only)

▶<contestant>

▶<country>

▶<venue>

▶<playerRatings>

▶<contestant>

▶<player>

▶<matchDataScore>

▶<indexScore>

▶<rawIndexScore>

▶<lineUp>

▶<player>

▶<teamOfficial>

Player Ratings explained - Subscores and Total Score

- **Total Score rating (indexScore):** A total score is given for each player under the `indexScore` attribute in the feed response - these scores will range from 20-100 and an "average" performance will score 60. The formula that works out this overall match rating has been developed by Opta's own team of data scientists in

conjunction with our award-winning Data Editorial team. Using each subcomponent, the total score is calculated according to the player's position in the match. Each position is weighted with the various subscores being more/less important depending on the position.

- **Subscores rating (rawIndexScore):** The formula to calculate the total/index score rating includes six subscores - these are given for each player under six `rawIndexScore` and `type` attributes.
The six rating types are Attacking, Defending, Goalkeeping, Passing, Possession, and Shooting. You can find an explanation of these subscore types in the table below.
- **Base rating:** Each player starts with no rating until they have actions assigned to them, however the default value is 60 in terms of adjusting for scale of once events are added.

SUBSCORE TYPE	DESCRIPTION
Attacking	This component attempts to quantify how much a player is contributing to their team's offensive output. Here, v
Defending	The defensive subscore incorporates two elements: • active - the active defensive element uses stats on defensive actions directly accumulated by • passive - the passive defensive element considers the attacking output of the other team as
Goalkeeping	The main (though not complete) component for evaluating a goalkeeper's performance. This includes performar
Passing	This component assesses a player's efficiency in circulating the ball within their team while in possession. Locati
Possession	This component attempts to measure a player's affect on their team's ability to retain and solidify possession. H
Shooting	Contrary to Attack, this component prizes efficiency. Shot location and type is taken into account by proxy (e.g. c